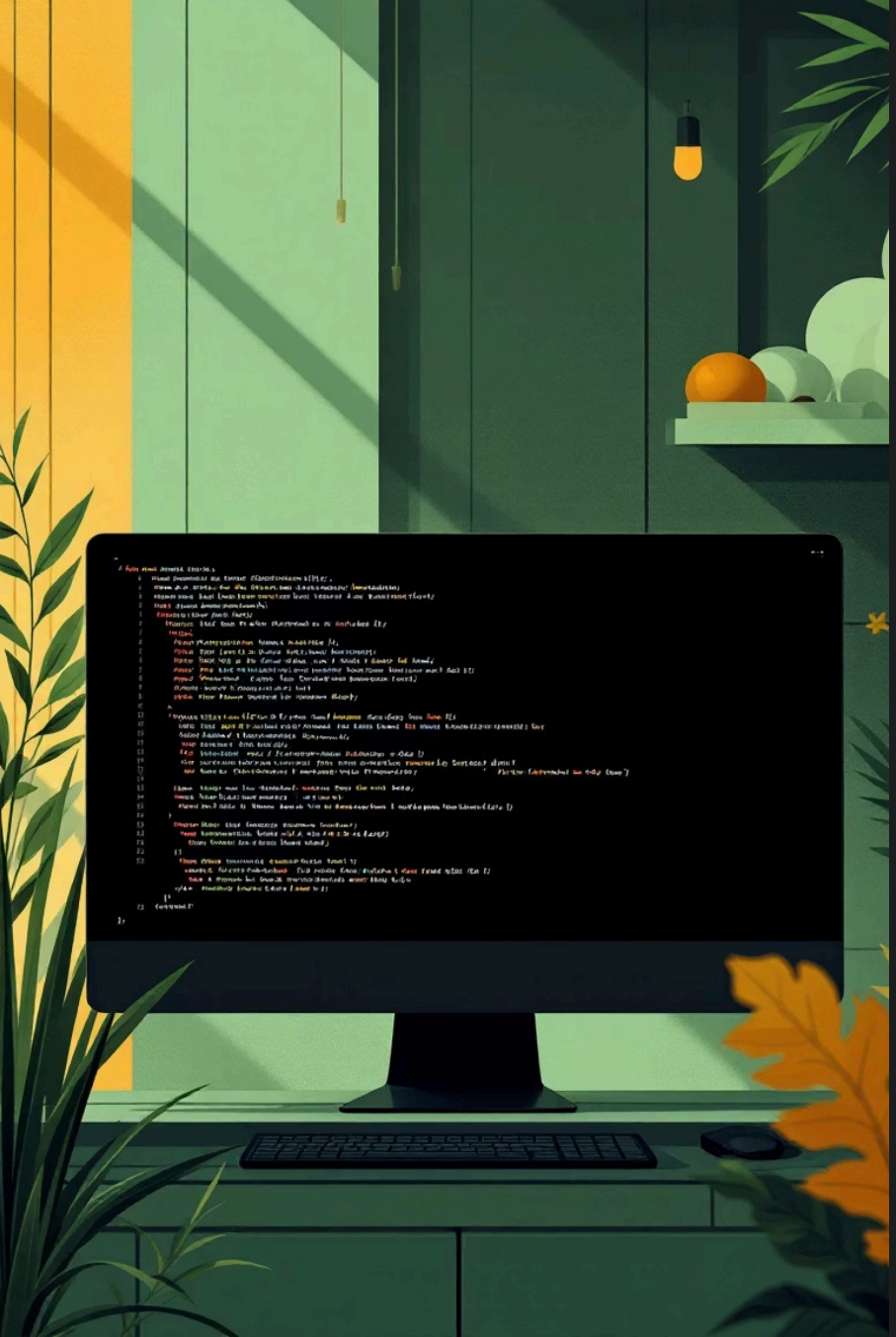


Automated Testing with Serenity BDD

This presentation will explore Serenity BDD, a powerful framework designed to streamline and enhance automated testing processes. We'll delve into its features, architecture, and practical applications, providing a comprehensive understanding for students of Software Engineering and Test Automation.

Introducing the Serenity BDD Framework

Serenity BDD (Behavior-Driven Development) is an open-source library that helps you write high-quality automated acceptance tests. It acts as a wrapper around popular testing tools, enriching their capabilities with powerful reporting and clear, narrative-driven test descriptions.

- **Brief History:** Evolved from the Thucydides framework, Serenity BDD has been continuously developed to meet the demands of modern software testing.
- **Test Pyramid Positioning:** While capable of supporting unit and API tests, Serenity BDD truly shines in the realm of UI and integration testing, providing comprehensive insights into user-facing functionalities.

Key Features of Serenity BDD

Automated & Visual Reports

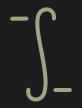
Generates detailed, narrative-style reports that are easy for both technical and non-technical stakeholders to understand, showcasing test outcomes and requirements coverage.

Step Reusability

Promotes the creation of reusable "Step Definitions," reducing code duplication and making tests more maintainable and readable across various scenarios.

Page Object Support

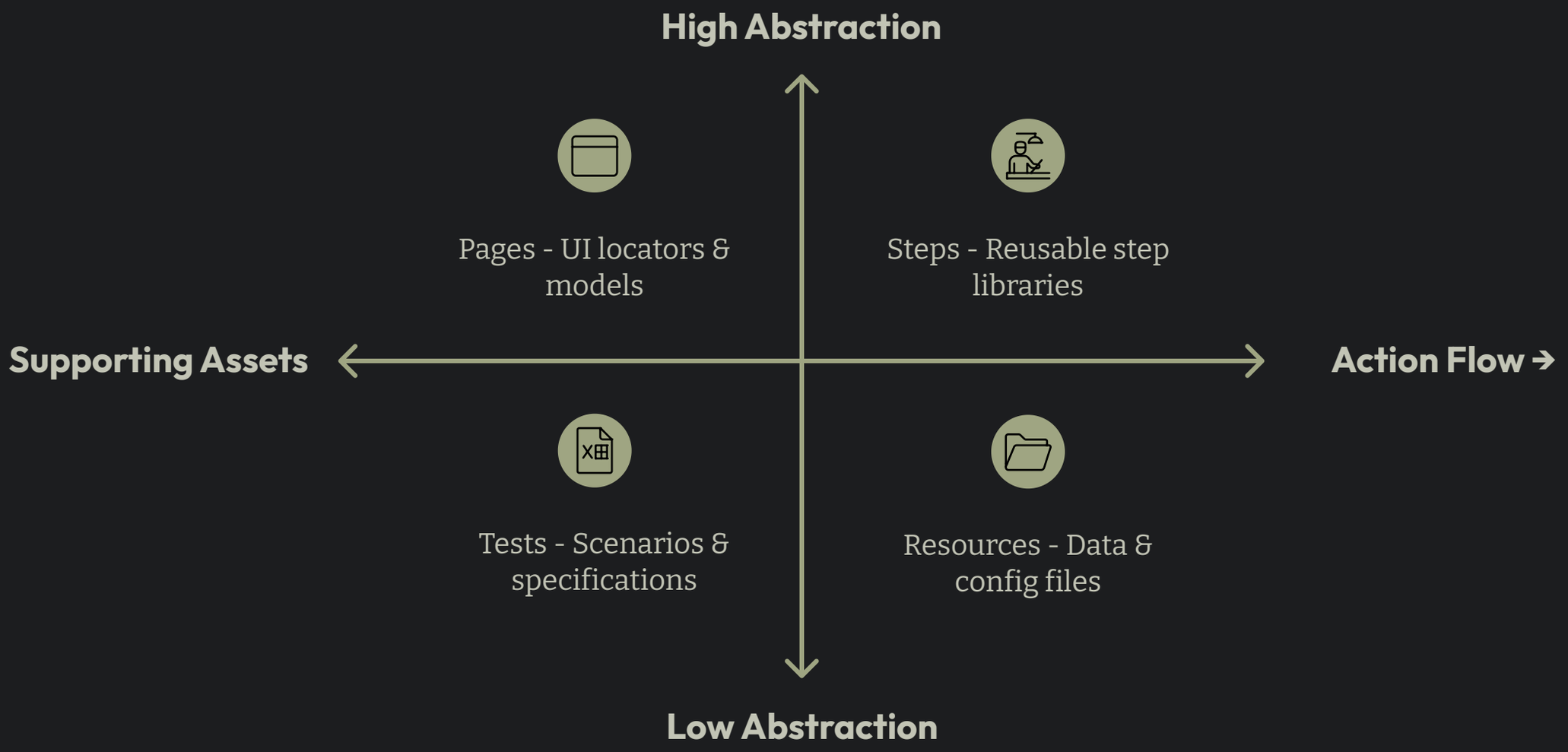
Strong support for the Page Object Model, a design pattern that makes UI tests more robust and easier to manage by abstracting page elements and interactions.



Seamless Integration

Integrates effortlessly with popular testing frameworks and tools like JUnit, Cucumber, and Selenium WebDriver, leveraging their strengths while adding its own reporting prowess.

Architecture and Project Structure

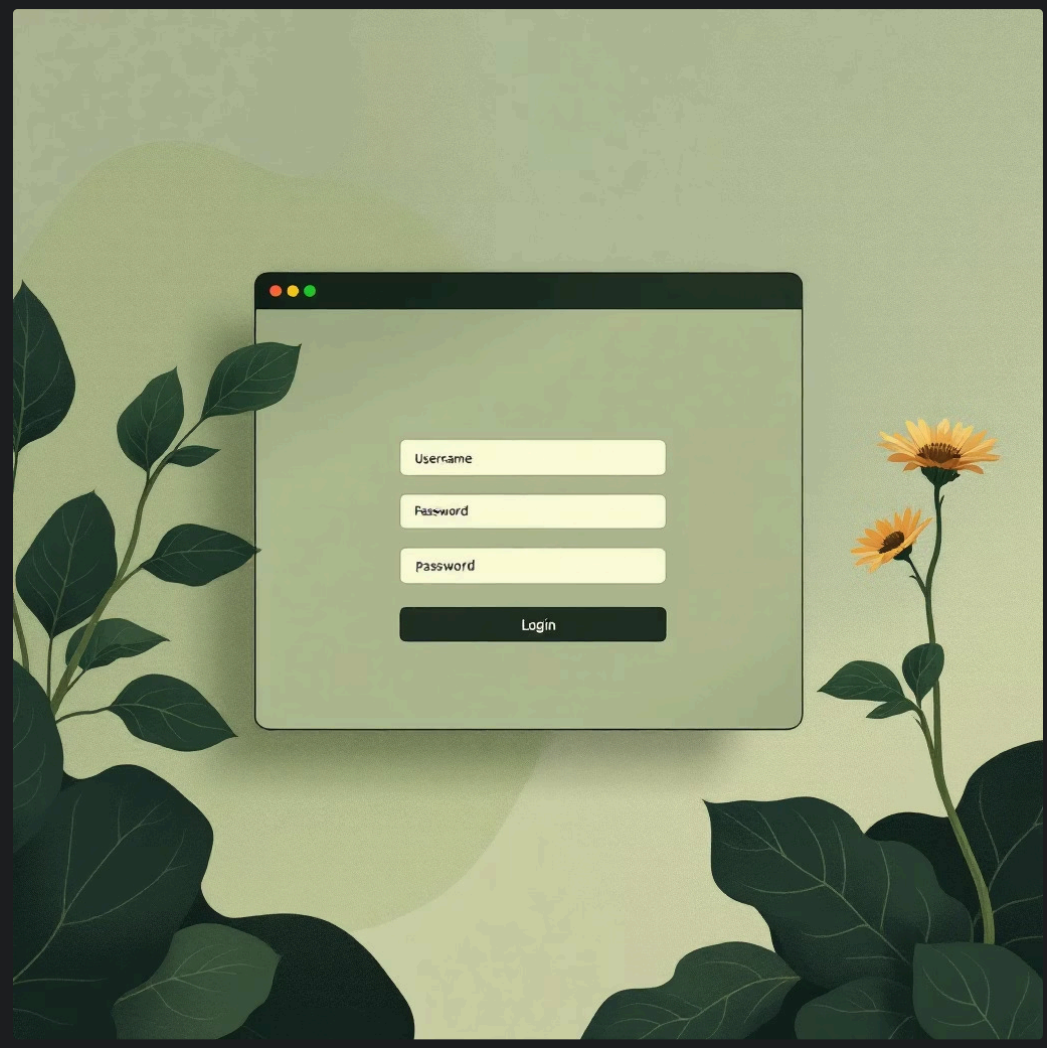


This organized structure offers significant benefits, including improved readability, easier maintenance, and better collaboration among team members.

Code Example: Page Object

The Page Object Model helps encapsulate UI elements and interactions. Here's a simplified example of how `LoginPage.java` might map elements on a login screen.

```
public class LoginPage extends PageObject {
    @FindBy(id="username")
    WebElementFacade usernameField;

    @FindBy(id="password")
    WebElementFacade passwordField;

    @FindBy(css=".login-button")
    WebElementFacade loginButton;

    public void enterUsername(String username) {
        usernameField.type(username);
    }

    public void enterPassword(String password) {
        passwordField.type(password);
    }

    public void clickLogin() {
        loginButton.click();
    }
}
```



This approach isolates UI changes, meaning if an element's locator changes, only the Page Object needs updating, not every test using that element.

Code Example: Steps

The `LoginSteps.java` class contains methods that represent user actions, acting as an abstraction layer over the Page Objects.

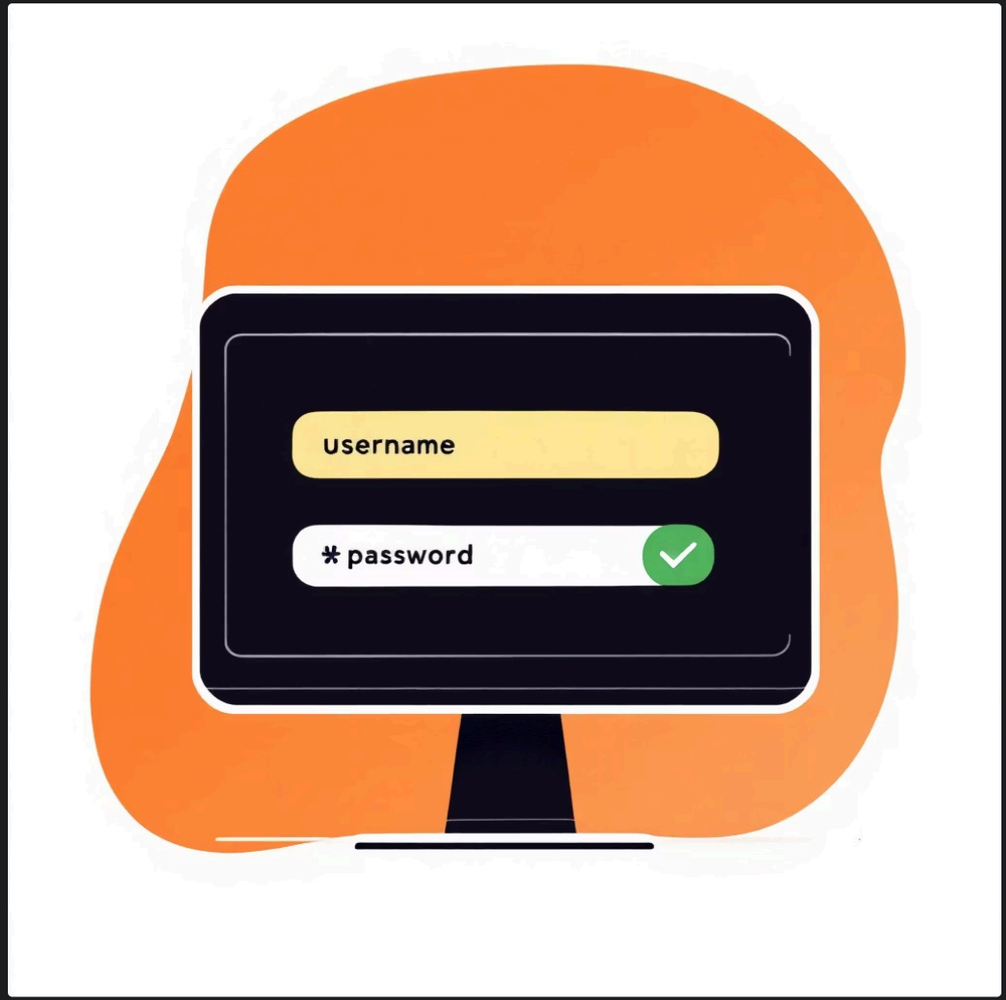
```
public class LoginSteps {
    LoginPage loginPage;

    @Step("Enters username: {0}")
    public void entersUsername(String username) {
        loginPage.enterUsername(username);
    }

    @Step("Enters password: {0}")
    public void entersPassword(String password) {
        loginPage.enterPassword(password);
    }

    @Step("Clicks login button")
    public void clicksLoginButton() {
        loginPage.clickLogin();
    }

    @Step("Logs in with username {0} and password {1}")
    public void logsIn(String username, String password) {
        entersUsername(username);
        entersPassword(password);
        clicksLoginButton();
    }
}
```

Each method annotated with `@Step` becomes a readable entry in the Serenity BDD report, providing a clear narrative of the test execution.

Code Example: Test

Finally, the `LoginTest.java` orchestrates the test scenario by calling the step methods.

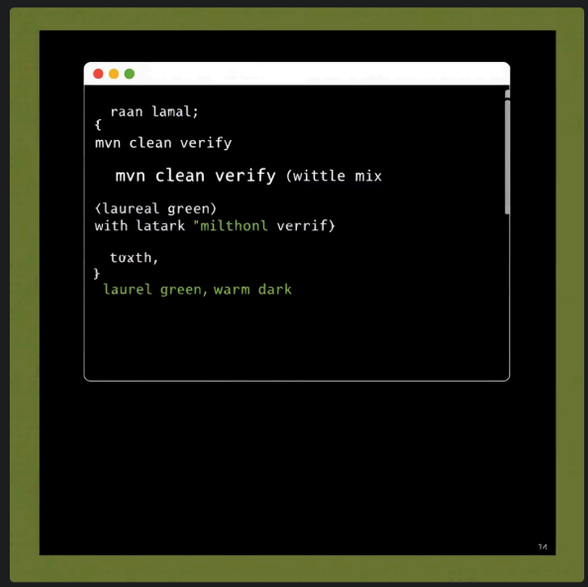
```
@RunWith(SerenityRunner.class)
public class LoginTest {

    @Managed(driver = "chrome")
    WebDriver driver;

    @Steps
    LoginSteps user;

    @Test
    public void userCanLoginSuccessfully() {
        driver.get("http://your-app-url.com/login");
        user.logsIn("validUser", "password123");
        // Assertions for successful login go here
    }
}
```



This test clearly describes the scenario: navigating to the login page, performing login actions, and then validating the outcome, all through concise and readable steps.

Execution and Reporting

Executing Serenity BDD tests is straightforward, typically managed through Maven.



- **How to Execute:** Simply run `mvn clean verify` from your project root. Maven will compile your tests, run them, and then generate the Serenity reports.
- **Report Location:** Once execution is complete, navigate to `target/site/serenity/index.html` in your project directory to view the comprehensive reports.

Example of a Generated Serenity Report: These reports include execution times, screenshots, detailed step-by-step breakdowns, and even videos of the test run, offering unparalleled visibility into your test suite.

Serenity BDD: Advantages and Considerations

Clear, Rich Reports

Provides highly detailed, narrative-driven reports that bridge the communication gap between technical and non-technical teams.

Enhanced Test Organization

Encourages good practices like the Page Object Model and step reusability, leading to more maintainable and scalable test suites.

Seamless Integrations

Works well with existing and popular tools like Selenium, Cucumber, and JUnit, leveraging their strengths.

Learning Curve

Adopting Serenity BDD might involve an initial learning curve, especially for teams unfamiliar with BDD principles or the specific framework structure.

Java/Maven Dependency

Primarily built for Java projects and integrates deeply with Maven, which might be a constraint for non-Java environments.

Conclusion and Recommendations

> Serenity BDD is an invaluable tool for teams aiming to achieve high-quality, readable, and maintainable automated tests. Its comprehensive reporting and structured approach foster better collaboration and clearer understanding of application behavior.

- **When to Adopt:** Ideal for projects requiring strong reporting, collaborative BDD, and complex UI automation, especially within the Java ecosystem.
- **Success Stories:** Many companies worldwide leverage Serenity BDD to enhance their software quality and accelerate their delivery pipelines.

Thank you for your attention. Are there any questions?